Compte rendu - TP2 : Exercice 3

Programmation orientée objet en C++ : héritage (classes **Personnel** et **Patron**)

1. Présentation

Cet exercice met en place une relation d'héritage : la classe **Patron** hérite publiquement de la classe **Personnel** et lui ajoute deux attributs, le bonus annuel (*ba*) et la marque de voiture (*mv*). Le programme principal teste la construction, la saisie, l'affichage, la modification et une fonction amie.

Fichier	Rôle
Personnel.h / Personnel.cpp	Classe de base : nom, position, salaire (attributs protégés), saisie, affichage, modification et fonction amie affiche_externe().
Patron.h / Patron.cpp	Classe dérivée de Personnel : ajoute ba et mv, avec ses constructeurs, saisie_patron() et affiche_patron().
exercice3.cpp	Programme principal : scénario de test.

2. Classe de base : Personnel

2.1 Personnel.h

Déclaration de la classe. Les attributs sont **protected** afin que la classe dérivée y accède ; *affiche_externe* est déclarée amie.

```
1  class Personnel {
2      protected:
3          string nom;
4          string position;
5          float salaire;
6      public:
7          Personnel();
8          Personnel(string nom, string position, float salaire) : nom(nom),
9                  position(position), salaire(salaire) {}
10         void affiche_personnel() const;
11         void saisie_personnel();
12         friend void affiche_externe(Personnel personnel);
13         void changer_salaire();
14         void changer_position();
15 };
```

Capture 1 - Fichier Personnel.h (corps de la classe)

2.2 Personnel.cpp

Implémentation des méthodes : affichage, constructeur par défaut, saisie, fonction amie, changement de salaire et de position.

```cpp
void Personnel::affiche_personnel() const {
    cout << "== personnel ==" << endl;
    cout << "nom: " << nom << endl;
    cout << "post: " << position << endl;
    cout << "salaire: " << salaire << endl;
}

Personnel::Personnel(): nom(""), position(""), salaire(0.0){

}

void Personnel::saisie_personnel(){
    cout << "nom: "; cin >> nom;
    cout << "post: "; cin >> position;
    cout << "salaire: "; cin >> salaire;
}

void affiche_externe(Personnel p){
    p.affiche_personnel();
    cout << endl;
}

void Personnel::changer_salaire(){
    cout << "nv salaire de "<< nom << " (" << position << "): "; cin >> salaire;
}

void Personnel::changer_position(){
    cout << "nv position de "<< nom << " (" << position << "): "; cin >> position;
}
```

Capture 2 - Fichier Personnel.cpp

3. Classe dérivée : Patron

3.1 Patron.h

Patron hérite publiquement de **Personnel** et ajoute les attributs *ba* et *mv*.

```cpp
class Patron: public Personnel {
    private:
        float ba;
        string mv;
    public:
        Patron(const string &nom, const string &position,const float &salaire,
                const float &ba, const string &mv);
        Patron();
        void saisie_patron();
        void affiche_patron() const;
};
```

Capture 3 - Fichier Patron.h

3.2 Patron.cpp

Le constructeur paramétré transmet nom, position et salaire au constructeur de Personnel. Le constructeur par défaut lance la saisie complète.

```cpp
Patron::Patron(const string &nom, const string &position,const float &salaire,
        const float &ba, const string &mv) : Personnel(nom, position, salaire),
        ba(ba), mv(mv){

}

Patron::Patron() {
    saisie_patron();
}

void Patron::saisie_patron(){
    saisie_personnel();
    cout << "===" << endl;
    cout << "bonus annuel: "; cin >> ba;
    cout << "marque de voiture: "; cin >> mv;
    cout << endl;
}

void Patron::affiche_patron() const{
    cout << "==== patron ====" << endl;
    affiche_personnel();
    cout << "===============" << endl;
    cout << "bonus annuel: " << ba << endl;
    cout << "marque de voiture: " << mv << endl;
    cout << endl;
}
```

Capture 4 - Fichier Patron.cpp

4. Programme de test

4.1 exercice3.cpp

Le scénario : création d'un patron avec le constructeur paramétré, création d'un second par saisie clavier, modification du salaire et de la position, puis appel de la fonction amie.

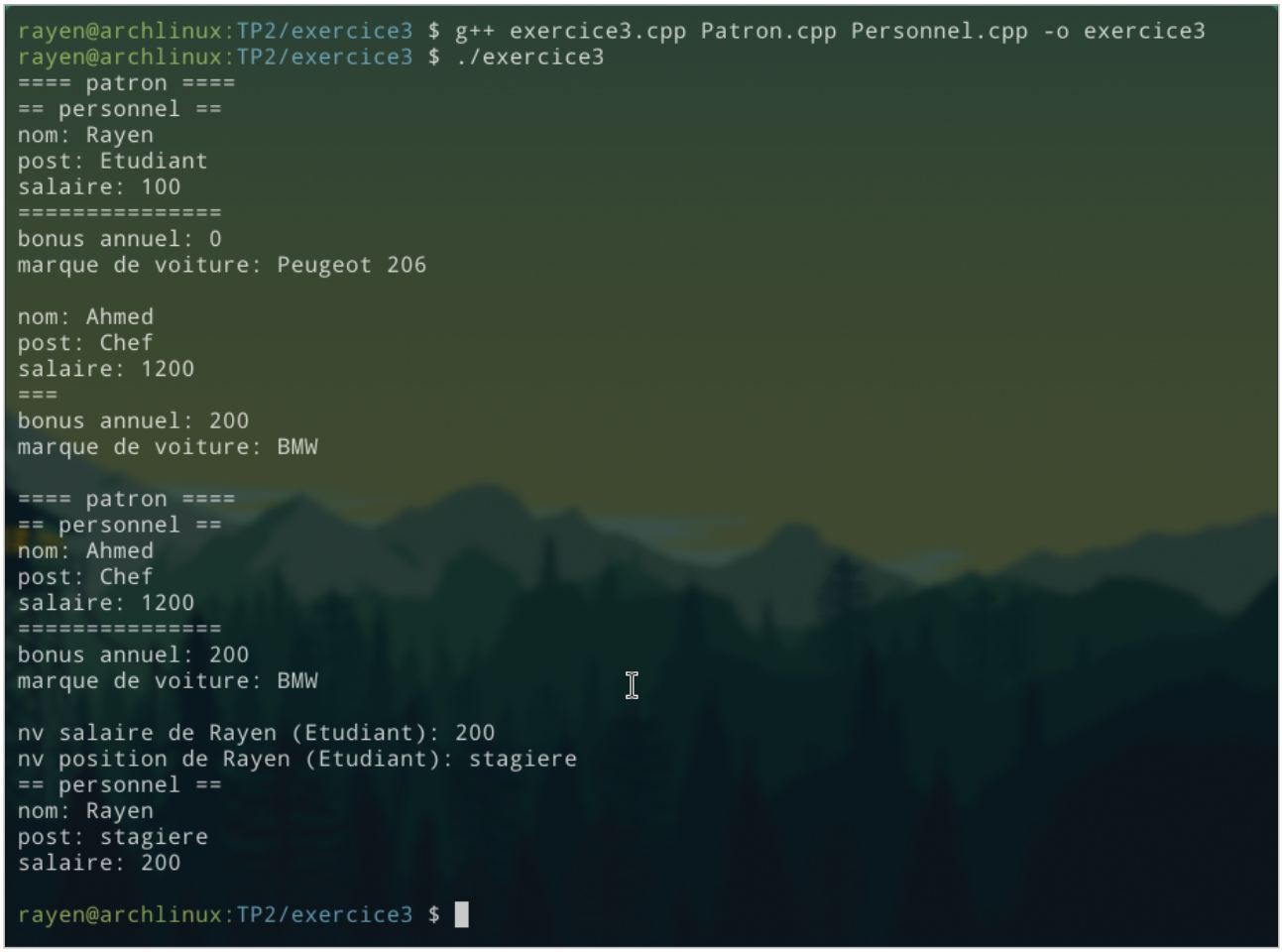
```cpp
int main() {
    Patron patron("Rayen", "Etudiant", 100, 0, "Peugeot 206");
    patron.affiche_patron();
    Patron p2;
    p2.affiche_patron();
    patron.changer_salaire();
    patron.changer_position();
    affiche_externe(patron);
    return 0;
}
```

Capture 5 - Fichier exercice3.cpp

5. Compilation et exécution

Compilation et lancement depuis le terminal :

```
g++ exercice3.cpp Patron.cpp Personnel.cpp -o exercice3
./exercice3
```



Capture 6 - Exécution du scénario de test

6. Analyse des résultats

- **Patron paramétré** : l'affichage montre d'abord la partie Personnel (Rayen, Etudiant, 100) puis la partie Patron (bonus annuel 0, voiture Peugeot 206).

- **Patron par défaut** : le constructeur appelle la saisie ; les valeurs entrées (Ahmed, Chef, 1200, 200, BMW) sont ensuite affichées correctement.

- **Modification** : *changer_salaire()* et *changer_position()*, héritées de Personnel, sont appelées directement sur l'objet Patron ; Rayen passe à un salaire de 200 et à la position « stagiere ».

- **Fonction amie** : *affiche_externe(patron)* reçoit un Personnel par valeur. L'objet est donc converti en Personnel (*slicing*) : seules les données de base sont affichées (nom, post, salaire), sans le bonus ni la voiture. Les valeurs modifiées sont bien visibles.

7. Conclusion

L'héritage permet de réutiliser le code de Personnel dans Patron sans le dupliquer. Le test confirme le bon fonctionnement des deux constructeurs, des méthodes héritées et de la fonction amie.